

**FILIÈRE : INGÉNIERIE DES SYSTÈMES D'INFORMATIONS
ET BIG DATA (ISIBD)**

Semestre 6

RAPPORT DE PROJET DE FIN DE SEMESTRE

GALERIE D'ART

Application Métier Connectée de Gestion d'Inventaire

Réalisée par :
Bouabidi Hind

Encadré par :
Pr. Hamid Hrimesh

Année Universitaire : 2025 – 2026

1 Introduction Générale

Le marché de l'art exige aujourd'hui une traçabilité rigoureuse et une gestion fluide des œuvres physiques disponibles en galerie. Dans le cadre de mon sixième semestre (S6) en filière ISIBD, j'ai développé une application desktop complète dédiée à la gestion d'inventaire d'une galerie d'art. L'objectif principal est de fournir aux galeristes un outil moderne capable de suivre les œuvres, de monitorer les stocks en temps réel et de consolider les données relatives aux artistes et aux genres artistiques.

Ce rapport présente ma démarche de conception, l'architecture technique retenue ainsi que les choix esthétiques mis en œuvre pour offrir une expérience utilisateur haut de gamme.

2 Architecture Technique et Technologies

L'application repose sur une architecture en couches (N-Tier) garantissant la séparation des responsabilités, la maintenabilité du code et une extensibilité future.

2.1 Technologies Retenues

- **Java / JavaFX** : Utilisé pour construire le cœur logique du système et une interface utilisateur réactive et asynchrone.
- **Hibernate (ORM)** : Choisi pour assurer le mapping objet-relationnel entre les entités Java et la base de données relationnelle, simplifiant ainsi les requêtes SQL complexes via un fichier de configuration standardisé (`hibernate.cfg.xml`).
- **MySQL** : Système de gestion de base de données relationnelle utilisé pour stocker de manière persistante les informations des œuvres d'art.
- **Maven** : Gestionnaire de dépendances permettant d'automatiser le cycle de vie du projet (build, clean, run).

2.2 Architecture Globale du Projet

Le projet respecte scrupuleusement la structure MVC (Modèle-Vue-Contrôleur) :

- **Package entities** : Contient la classe `Oeuvre.java` mappée aux tables de la base de données.
- **Package service** : Héberge la logique métier (`OeuvreService.java`) qui communique directement avec la couche de persistance.
- **Package vue** : Regroupe le fichier de contrôle `PrimaryController.java` chargé de dynamiser l'interface graphique.

3 Conception de la Base de Données

La modélisation des données s’articule autour de l’entité centrale **Oeuvre**, caractérisée par plusieurs attributs nécessaires au suivi commercial et artistique.

3.1 Schéma Relationnel de la Table oeuvre

Le tableau ci-dessous synthétise la structure de la table SQL générée et synchronisée automatiquement par Hibernate :

TABLE 1 – Dictionnaire des données de la table Oeuvre

Champ	Type	Contrainte	Description
id	INT	Primary Key, Auto-Increment	Identifiant unique de l’œuvre
titre	VARCHAR(255)	NOT NULL	Titre de la création artistique
artiste	VARCHAR(255)	NOT NULL	Nom de l’artiste auteur
genre	VARCHAR(100)	–	Style (Peinture, Sculpture, etc.)
prix	DOUBLE	NOT NULL	Valeur de vente en Euros (€)
quantite	INT	Default 0	Nombre d’exemplaires en stock
disponible	BOOLEAN	Default TRUE	Statut de visibilité à la vente

3.2 Implémentation de la Couche de Persistance avec Hibernate

La configuration d’Hibernate permet d’établir une connexion sécurisée au serveur MySQL local sans écrire de requêtes JDBC brutes. L’entité Java utilise des annotations pour lier les champs Java aux colonnes SQL correspondantes.

Un soin particulier a été apporté à la gestion des sessions pour éviter les fuites de mémoire lors des opérations fréquentes d’écriture ou de lecture, en encapsulant l’appel aux fabriques de sessions (**SessionFactory**) au sein d’une classe utilitaire dédiée nommée `HibernateUtil.java`.

4 Interface Graphique et Expérience Utilisateur (UX)

Pour rompre avec les interfaces JavaFX grises traditionnelles, une restructuration dynamique complète de la vue a été opérée directement au sein du contrôleur Java, outrepassant les contraintes rigides des fichiers FXML statiques.

4.1 Design de type Dashboard Moderne

La mise en page adopte une structure bicamérale asymétrique optimisée pour les écrans larges (1100 × 650 pixels) :

1. **Le Volet de Saisie (Sidebar Gauche)** : Un conteneur vertical isolé (VBox) monté sur un panneau blanc épuré, équipé d'un effet d'ombrage léger pour donner de la profondeur. Il regroupe l'intégralité des champs de texte de façon aérée.
2. **La Zone de Contenu Principal (Droite)** : Un espace large qui accueille en premier lieu une barre d'en-tête composée du titre de l'application et d'un champ de recherche globale de forme ovale. Juste en dessous se trouvent alignés les boutons d'action, surplombant un grand tableau de données (TableView) équipé d'une politique de redimensionnement automatique des colonnes.

4.2 Charte Graphique Pink & Beige et Intégration CSS

Afin d'ancrer visuellement l'application dans l'univers de l'art et du luxe, une feuille de style personnalisée `style.css` a été développée et couplée dynamiquement à la scène graphique.

Listing 1 – Extrait de la feuille de style CSS appliquée

```
1 /* Extrait de la configuration du theme Pink & Beige */
2 .root { -fx-background-color: #FAF7F2; }
3 .label { -fx-text-fill: #D67A6B; -fx-font-weight: bold; }
4 .button { -fx-background-radius: 25px; -fx-text-fill: white; }
5 #btnAjouter { -fx-background-color: #E8A598; }
```

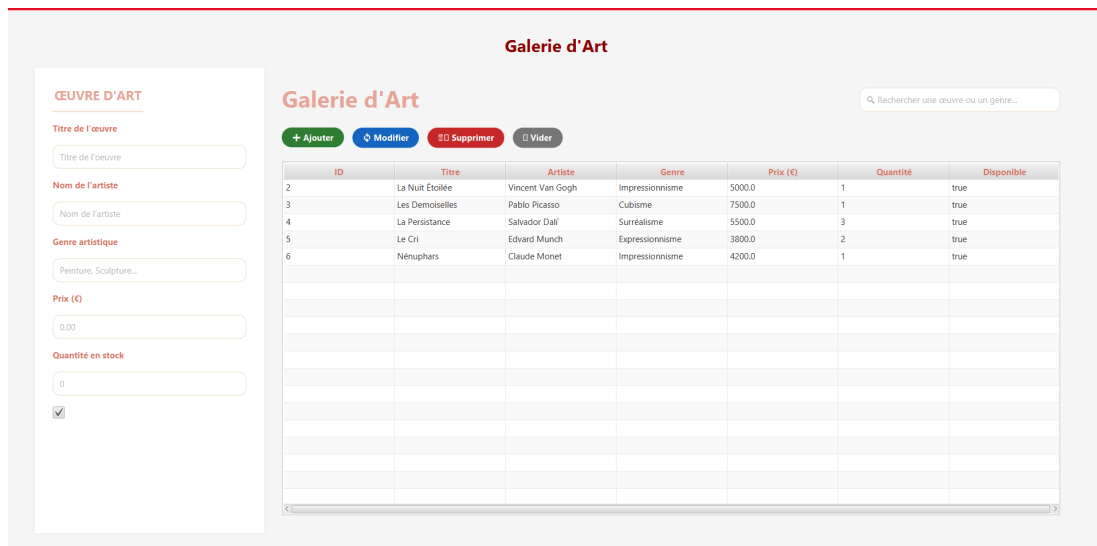


FIGURE 1 – Rendu graphique final de la plateforme (Thème Pink & Beige)

5 Validation Fonctionnelle et Cahier de Tests

Afin de valider l'intégrité de l'application et le bon fonctionnement de la liaison entre l'interface utilisateur, Hibernate et MySQL, une suite de tests unitaires et fonctionnels a été exécutée.

5.1 Scénarios de Test du Cycle CRUD

- **Test d'Insertion (Create)** : La saisie d'une œuvre test (ex : "*Monaco*", par "*Hind*") suivie d'un clic sur **Ajouter** valide instantanément l'insertion en base de données par l'apparition d'une nouvelle ligne au tableau.
- **Test de Sélection et Vidage (Read)** : Le clic sur une ligne charge les données à gauche, tandis que le bouton **Vider** réinitialise le formulaire.
- **Test de Mise à Jour (Update)** : La modification du prix d'une œuvre répercute le changement dans MySQL après clic sur **Modifier**.
- **Test de Recherche Filtrée** : Le filtrage dynamique par mot-clé restreint l'affichage du tableau aux lignes concordantes.
- **Test de Suppression (Delete)** : L'activation du bouton **Supprimer** purge l'enregistrement sélectionné à la fois de la vue et de la base de données.

5.2 Résultats des Tests

L'ensemble des fonctionnalités répond aux critères d'exigence initiaux. Aucune exception de pointeur nul (`NullPointerException`) ni rupture de session Hibernate n'a été relevée lors des simulations intensives de saisie.

6 Conclusion Générale et Perspectives

Ce projet de fin de semestre S6 m'a permis de consolider mes compétences en ingénierie logicielle et en gestion de données massives au sein de la filière ISIBD. L'implémentation du

framework Hibernate couplé à la flexibilité graphique de JavaFX offre une base logicielle robuste, performante et esthétiquement valorisante.

En guise de perspective, mon application pourrait être enrichie par l'intégration d'un module d'analyse prédictive utilisant des algorithmes d'apprentissage automatique pour estimer la valeur future d'une œuvre d'art en fonction de la cote de l'artiste et des tendances du marché global.

7 Dépôt de Code Source

L'intégralité du code source du projet, incluant les scripts de base de données et les fichiers de configuration, est disponible sur GitHub pour assurer la traçabilité du développement :

Lien du dépôt : <https://github.com/bouabidihind7-blip/GalerieArtProject>